

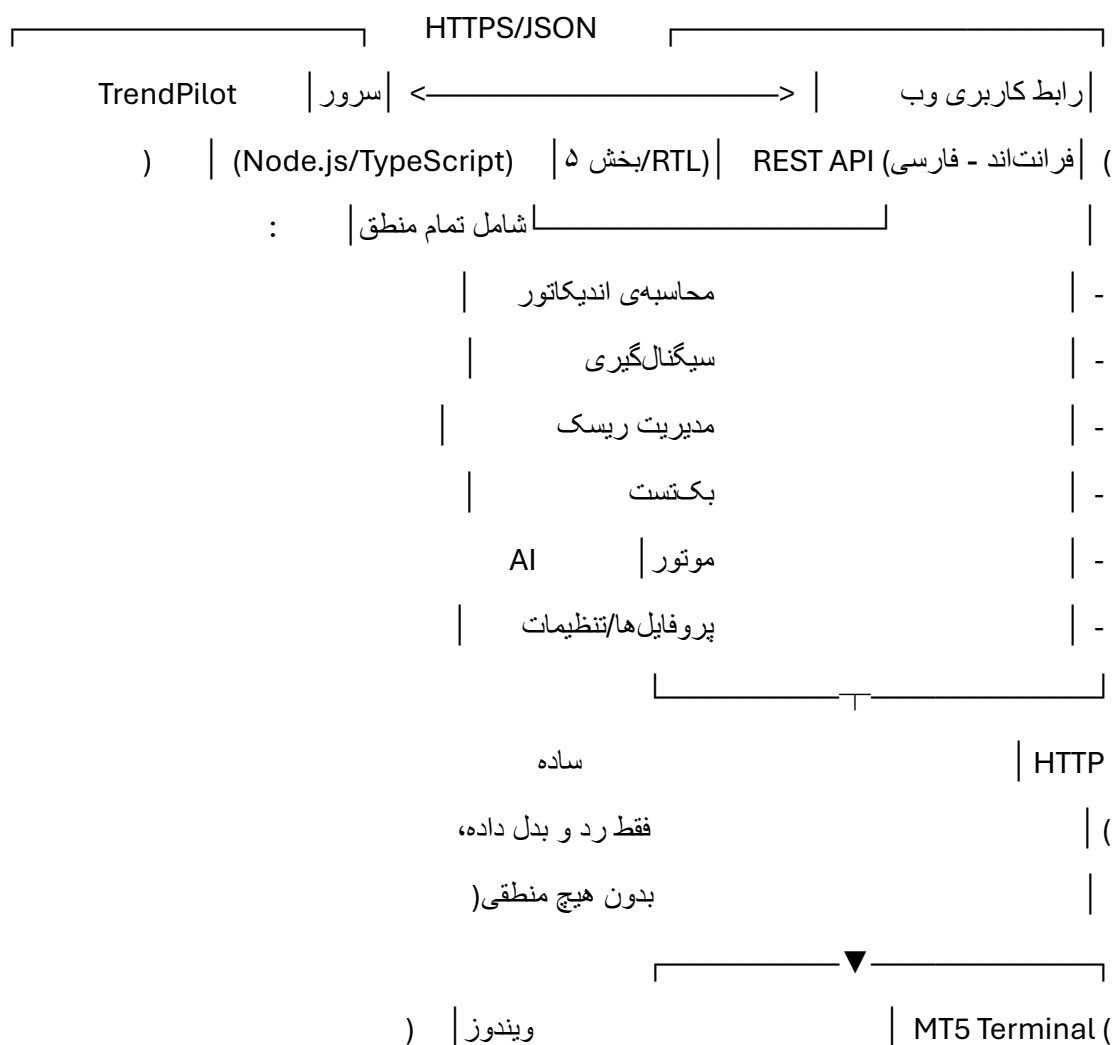
TrendPilot Web — مشخصات کامل ساخت اپلیکیشن

۱. این اپلیکیشن چیست

TrendPilot یک اپلیکیشن تحت وب برای مدیریت و اجرای ربات معاملاتی الگوریتمی روی پلتفرم MetaTrader 5 (MT5) است. کاربر از طریق یک رابط کاربری وب فارسی RTL می‌تواند:

- استراتژی معاملاتی را تنظیم کند (روند، اسکالپ، یا خودکار)
- چند «پرو فایل» معاملاتی مستقل همزمان تعریف و اجرا کند
- پوزیشن‌های باز، لاگ زنده، و گزارش عملکرد را ببیند
- روی داده‌ی تاریخی بکتست بگیرد
- یک موتور هوش مصنوعی کمکی را آموزش دهد و وضعیتش را ببیند

۲. معماری کلی سیستم



+	یک اسکریپت پل بسیار
	سبک (Expert Advisor)

اصل طراحی مهم: هیچ منطق تصمیم‌گیری یا محاسبه‌ای نباید در سمت MT5/ویندوز باشد. تنها کاری که آن لایه انجام می‌دهد این است: قیمت‌های لحظه‌ای/کندل را به سرور بفرستد، و دستور معامله‌ای که از سرور برمی‌گردد را اجرا کند. تمام هوش سیستم روی سرور **TypeScript** است، نه پایتون و نه اسکریپت داخل MT5. این طراحی باعث می‌شود:

- توسعه و دیباگ کردن فقط در یک محیط (سرور + رابط کاربری، هر دو TypeScript) انجام شود.
- سرور بتواند روی هر هاستی (حتی سرور ابری لینوکسی معمولی) میزبانی شود.

۳. رابط کاربری — تب‌ها و صفحات (آنچه باید ساخته شود)

زبان: فارسی، جهت راست‌به‌چپ (RTL) کامل. ظاهر: تیره، حرفه‌ای، مناسب نمایش داده‌های مالی (اعداد، جدول، نمودار). ریسپانسیو (قابل استفاده روی موبایل هم باشد).

۳.۱ داشبورد

خلاصه‌ی وضعیت کلی: بالانس و اکوییتی حساب، تعداد ربات‌های در حال اجرا، تعداد پوزیشن باز، سود/زیان روز جاری، وضعیت اتصال به MT5.

۳.۲ تنظیمات (پرو فایل‌ها)

فرم کامل تنظیم پارامترهای ربات (ساختار کامل در بخش ۶) با گروهبندی منطقی:

- اتصال MT5 (لاگین، پسورد، سرور)
- انتخاب نمادها و تایم‌فریم
- انتخاب حالت استراتژی (روند / اسکالپ / خودکار)
- تنظیمات ورود روند (ADX)، کول‌داون، حداکثر معاملات هم‌زمان
- تنظیمات ورود اسکالپ (بولینگر، RSI سریع، رد شدن کندل)
- فیلتر رژیم بازار (برای حالت خودکار)
- مدیریت ریسک (درصد ریسک هر معامله، SL/TP، اهرم)
- تأیید تایم‌فریم بالاتر (HTF)
- خروج (Trailing Stop)، حداکثر مدت معامله، بستن آخر هفته)
- محدودیت‌های کلی (سقف ریسک پرتفوی، سقف ضرر روزانه)

امکانات: ذخیره، کپی، حذف چند پروفایل مستقل؛ هر پروفایل یک «Magic Number» یکتا دارد تا معاملات پروفایل‌های مختلف از هم تفکیک شوند.

۳.۳ پوزیشن‌ها

جدول پوزیشن‌های باز، گروه‌بندی‌شده بر اساس پروفایل و نماد. ستون‌ها: نماد، نوع (خرید/فروش)، حجم، قیمت باز شدن، قیمت فعلی، SL، TP، سود لحظه‌ای، مارجین.

۳.۴ بکتست

فرم انتخاب: نماد(ها)، بازه‌ی زمانی (یا برچسب دوره‌ی از پیش‌تعریف‌شده مثل «۳ ماه اخیر»، یا تاریخ شروع/پایان دستی). اجرای بکتست به‌صورت Job غیرهمزمان (چون ممکن است طول بکشد) — کاربر وضعیت Job را چک می‌کند تا تمام شود. نمایش نتیجه: نمودار equity curve، جدول کامل معاملات (با دلیل خروج، ATR/ADX/RSI لحظه‌ی ورود، مدت نگهداری)، آمار کلی (سود کل، تعداد برد/باخت، Win Rate، حداکثر افت سرمایه، Profit Factor)، و آمار به‌تفکیک هر نماد. امکان خروجی گرفتن اکسل. (CSV/)

۳.۵ گزارش

گزارش عملکرد واقعی حساب (نه بکتست) در یک بازه‌ی زمانی دلخواه: سود کل، تعداد معاملات، Win Rate، فهرست معاملات بسته‌شده.

۳.۶ لاگ

نمایش زنده‌ی خطوط لاگ سرور (به‌روزرسانی دوره‌ای. (polling/)

۳.۷ موتور هوش مصنوعی (AI Engine)

نمایش وضعیت مدل فعلی (آموزش‌دیده یا نه، دقت). دکمه‌ی «آموزش مدل» با انتخاب نماد، تایم‌فریم، و تعداد کندل تاریخی برای آموزش.

۳.۸ درباره

توضیح مختصر پروژه و نسخه.

نکته برای پیاده‌سازی اولیه‌ی UI

چون سرور واقعی (بخش منطق معاملاتی) هنوز کامل نیست، رابط کاربری باید:

- آدرس پایه‌ی API را از یک متغیر محیطی/تنظیمات قابل‌تغییر بخواند (نه hardcode)، تا بعداً به‌راحتی به سرور واقعی وصل شود.
 - در نبود پاسخ واقعی از سرور، بتواند با داده‌ی نمونه (mock) کار کند تا ظاهر و تعامل UI قابل بررسی باشد.
 - نباید خودش منطق معاملاتی/محاسباتی (سیگنال‌گیری، مدیریت ریسک، بکتست) را پیاده‌سازی کند — این فقط UI است؛ تمام هوش سیستم در سرور جداگانه‌ای طبق بخش ۴ و ۵ ساخته می‌شود.
-

۴. سرور (بک‌اند) — چه چیزی باید ساخته شود

پشته‌ی پیشنهادی **Node.js + TypeScript**؛ فریم‌ورک وب سبک (مثل Fastify یا Express)، اعتبارسنجی ورودی (مثل Zod).

ماژول‌های اصلی سرور

1. **موتور اندیکاتور** — محاسبه‌ی $EMA(20)$ ، $EMA(50)$ ، RSI با صاف‌سازی وایلدِر (دوره‌ی ۱۴)، ATR با صاف‌سازی وایلدِر، $ADX/+DI/-DI$ با صاف‌سازی وایلدِر، کانال دانچیان، باند بولینگر، ضریب کارایی کافمن (Efficiency Ratio)، صدک ATR اخیر. ورودی: آرایه‌ای از کندل‌های OHLC خروجی: همان آرایه به‌همراه مقادیر اندیکاتور برای هر کندل. این ماژول باید کاملاً خالص باشد — فقط روی آرایه‌ی عددی کار کند، هیچ وابستگی به $MT5$ یا هر منبع داده‌ی خاصی نداشته باشد (قابل تست با داده‌ی مصنوعی).
2. **موتور سیگنال روند** — بر اساس کراس EMA سریع/کند، تأیید با ADX بالای یک آستانه یعنی روند معتبر)، فیلتر RSI ، و در صورت فعال بودن، تأیید از یک تایم‌فریم بالاتر — (HTF) یعنی سیگنال فقط وقتی معتبر است که روند در تایم‌فریم بزرگ‌تر هم هم‌جهت باشد.
3. **موتور سیگنال اسکالپ** — استراتژی برگشت‌ازحد (Mean-Reversion) ورود وقتی قیمت به باند بیرونی بولینگر می‌رسد و RSI سریع (دوره‌ی کوتاه‌تر) اشباع خرید/فروش نشان می‌دهد، با شرط اختیاری نیاز به کندل رد شدن (فنیله‌دار) قبل از ورود.
4. **تشخیص رژیم بازار** — برای حالت «خودکار»: بر اساس ADX پایین، شیب کم میانه‌ی بولینگر، ضریب کارایی کافمن پایین، و ATR در بازه‌ی صدکی میانی، بازار «رنج» تشخیص داده می‌شود و اگر رنج بود از موتور اسکالپ، وگرنه از موتور روند استفاده می‌شود.
5. **مدیریت ریسک** — محاسبه‌ی حجم معامله بر اساس درصد ریسک انتخابی و فاصله‌ی SL ؛ چند روش تعیین SL/TP قابل انتخاب (درصدی ساده، ساختاری بر اساس سطوح حمایت/مقاومت اخیر، یا بر اساس کانال دانچیان)؛ در نظر گرفتن اهرم حساب اگر «ریسک آگاه از اهرم» فعال باشد.
6. **مدیریت پوزیشن باز (Trailing Stop)** — فعال‌سازی و فاصله بر اساس ضریبی از ATR ، بستن اجباری بعد از مدت مشخص، بستن همه‌ی پوزیشن‌ها قبل از تعطیلی آخر هفته، و $kill-switch$ وقتی ضرر روزانه از سقف مشخصی بگذرد.
7. **پشتیبانی چند-پروفایلی هم‌زمان** — هر پروفایل مجموعه‌ی نماد و $Magic\ Number$ مستقل خودش را دارد؛ محدودیت سقف ریسک کل در سطح کل حساب (نه فقط هر پروفایل) هم باید رعایت شود.
8. **موتور بکتست** — روی داده‌ی تاریخی OHLC، همان منطق سیگنال/ریسک/خروج بالا را شبیه‌سازی می‌کند و گزارش کامل (equity curve)، معاملات، آمار (تولید می‌کند. باید دقیقاً همان منطقی را اجرا کند که در حالت زنده اجرا می‌شود) (یعنی کد سیگنال/ریسک بین بکتست و اجرای زنده مشترک باشد، نه دو پیاده‌سازی جدا که ممکن است ناهم‌خوان شوند).
9. **موتور هوش مصنوعی کمکی** — استخراج ویژگی از داده‌ی قیمتی EMA (فاصله، RSI ، ATR ، عرض بولینگر، نسبت بدنه‌ی کندل)، آموزش یک مدل ساده‌ی طبقه‌بندی (مثل رگرسیون لجستیک یا یک شبکه‌ی کوچک) برای پیش‌بینی جهت حرکت بعدی، ذخیره/بارگذاری مدل، و یک $endpoint$ برای گرفتن پیش‌بینی لحظه‌ای.

10. **پروفایل‌ها و تنظیمات** — ذخیره‌سازی پایدار (فایل JSON یا دیتابیس سبک مثل SQLite) برای پروفایل‌های ربات و گروه‌های نماد.

11. **API عمومی** — تمام endpoint های بخش ۷ را پیاده‌سازی می‌کند.

12. **یک endpoint داخلی مخصوص پل MT5** مثلاً: (POST /api/v1/analyze): کندل‌های اخیر یک نماد را می‌گیرد، اندیکاتور + سیگنال + دستور نهایی (خرید/فروش/بستن/نگه‌داشتن + حجم SL/TP) را برمی‌گرداند. این تنها endpointی است که پل سمت MT5 صدا می‌زند.

۵. لایه‌ی پل سمت MT5 تنها بخش وابسته به ویندوز)

یک **Expert Advisor** بسیار سبک به زبان (MQL5 زبان بومی MT5، نه پایتون) که داخل خود ترمینال MT5 اجرا می‌شود. وظایفش:

1. هر چند ثانیه یکبار، آخرین N کندل نماد فعلی را به‌صورت JSON با HTTP POST به سرور (endpoint) بخش ۴.۱۲ (می‌فرستد).

2. پاسخ JSON سرور (شامل دستور معامله) را می‌خواند.

3. اگر دستور خرید/فروش/بستن بود، همان لحظه با تابع بومی (OrderSend) MT5 آن را اجرا می‌کند.

این EA هیچ محاسبه یا تصمیمی نمی‌گیرد — فقط یک واسط داده‌ست. حجم کدش باید بسیار کم باشد (چند ده خط). برای این‌که MT5 اجازه‌ی HTTP request بدهد، آدرس سرور باید در تنظیمات ترمینال (Tools > Options > Expert Advisors > Allow WebRequest) به‌صورت صریح مجاز شود.

۶. مدل کامل تنظیمات ربات (BotConfig)

```
interface BotConfig {  
    // اتصال به MT5  
    MT5_LOGIN: number;  
    MT5_PASSWORD: string;  
    MT5_SERVER: string;  
    MT5_PATH: string | null;  
  
    // نمادها و تایم‌فریم  
    SYMBOLS: string[];
```

1440 ,240 ,60 ,30 ,15 ,5 ,1 TIMEFRAME: number; //

STRATEGY_MODE: string; // " روند" | "اسکلپ (Trend Following)" | "خودکار (Scalping)" | "تشخیص رژیم بازار"

// فیلتر تشخیص رژیم بازار (روند در برابر رنج)

REGIME_ADX_MAX: number;

REGIME_BB_SLOPE_LOOKBACK: number;

REGIME_BB_SLOPE_MAX_PERCENT: number;

REGIME_EFFICIENCY_LOOKBACK: number;

REGIME_EFFICIENCY_MAX: number;

REGIME_ATR_LOOKBACK: number;

REGIME_ATR_MIN_PERCENTILE: number;

REGIME_ATR_MAX_PERCENTILE: number;

// ورود اسکلپ (برگشت از حد)

SCALP_BB_PERIOD: number;

SCALP_BB_STD: number;

SCALP_RSI_PERIOD: number;

SCALP_RSI_OVERSOLD: number;

SCALP_RSI_OVERBOUGHT: number;

SCALP_REQUIRE_REJECTION: boolean;

SCALP_REJECTION_WICK_RATIO: number;

// خروج اسکلپ

SCALP_STRUCTURE_LOOKBACK: number;

SCALP_STRUCTURE_BUFFER_ATR: number;

SCALP_ATR_SL_MULTIPLIER: number;
SCALP_TP_MODE: string; // " (TP1) باند | " (TP2) خودکار (بر اساس قدرت حرکت) "
SCALP_TP_ATR_FALLBACK: number;
SCALP_MIN_RR: number;
SCALP_MAX_SPREAD_POINTS: number;
SCALP_SESSION_FILTER_ENABLED: boolean;
SCALP_SESSION_START_HOUR_UTC: number;
SCALP_SESSION_END_HOUR_UTC: number;

// مدیریت ریسک / حد ضرر-سود

RISK_PERCENT_PER_TRADE: number;
SL_PERCENT: number;
TP_PERCENT: number;
ENABLE_LEVERAGE_AWARE_PERCENT_STOPS: boolean;
ENABLE_SR_STOPS: boolean;
SR_LOOKBACK_BARS: number;
SR_BUFFER_ATR_MULTIPLIER: number;
ENABLE_DONCHIAN_SL: boolean;
DONCHIAN_PERIOD: number;
DONCHIAN_ATR_MULTIPLIER: number;
RISK_TOLERANCE_PROFILE: string;

// ورود روند

ADX_TREND_THRESHOLD: number;
COOLDOWN_BARS: number;
MAX_CONCURRENT_TRADES: number;

MAX_TOTAL_RISK_PERCENT: number;
ENABLE_DAILY_LOSS_LIMIT: boolean;
DAILY_LOSS_LIMIT_PERCENT: number;
ENABLE_HTF_CONFIRMATION: boolean;
HTF_TIMEFRAME: number | null;

// خروج

ENABLE_TRAILING_STOP: boolean;
TRAILING_ACTIVATION_ATR_MULTIPLIER: number;
TRAILING_DISTANCE_ATR_MULTIPLIER: number;
ENABLE_MAX_TRADE_DURATION: boolean;
MAX_TRADE_DURATION_HOURS: number;
ENABLE_WEEKEND_CLOSE: boolean;
WEEKEND_CLOSE_HOUR_UTC: number;

// متفرقه

REQUIRE_DEMO_ACCOUNT: boolean;
PROFILE_NAME: string;
MAGIC_NUMBER: number;
INITIAL_BALANCE?: number;
}

مدل‌های داده‌ی مرتبط دیگر

interface Position {
 ticket: number;
 symbol: string;

type: number; // 0 = خرید، 1 = فروش


```
        volume: number;
        price_open: number;
        price_current: number;
        sl: number;
        tp: number;
        profit: number;
        magic: number;
        margin: number;
        leverage: number;
    }
```

```
    interface BotStatus {
        connected: boolean;
        is_running: boolean;
        profile_name: string;
        magic_number: number;
        balance?: number;
        equity?: number;
        currency?: string;
        account_leverage?: number | null;
        open_positions?: Position[];
        rejected_signals_total: number;
        rejected_signals_by_symbol: Record<string, number>;
        rejected_signals_by_code: Record<string, number>;
        last_rejection: Record<string, any>;
        symbol_states: Record<string, any>;
    }
```

```
symbol_issues?: Record<string, string>;  
    configured_symbols?: string[];  
    bot_open_trades_count?: number;  
    open_risk_percent?: number;  
    daily_loss_triggered: boolean;  
    }
```

```
interface ReportTrade {  
    ticket: number;  
    symbol: string;  
    type: string;  
    volume: number;  
    price: number;  
    profit: number;  
    time: string;  
    }
```

```
interface PerformanceReport {  
    error?: string;  
    total_profit: number;  
    total_trades: number;  
    win_rate: number;  
    wins: number;  
    losses: number;  
    trades: ReportTrade[];  
    account_login?: number;
```

```
account_server?: string;
    balance?: number;
    equity?: number;
    currency?: string;
    }
```

```
interface BacktestTrade {
    ticket: number;
    symbol: string;
    side: string;
    volume: number;
    open_time: string;
    open_price: number;
    sl: number;
    tp: number;
    close_time?: string;
    close_price?: number;
    profit: number;
    commission: number;
    exit_reason: string;
    exit_reason_fa: string;
    atr_at_open: number;
    adx_at_open: number;
    rsi_at_open: number;
    htf_trend: string;
    sl_tp_method: string;
```

```
bars_held: number;  
}
```

```
interface BacktestResultPayload {  
    total_profit: number;  
    wins: number;  
    losses: number;  
    win_rate: number;  
    max_dd_pct: number;  
    max_dd_money: number;  
    advanced: Record<string, any>;  
    tips: string[];  
    trades: BacktestTrade[];  
    per_symbol: {  
        symbol: string;  
        total_profit: number;  
        total_trades: number;  
        win_rate: number;  
        profit_factor: number;  
    };  
    max_drawdown_percent: number;  
    expectancy: number;  
};  
}
```

```
interface BacktestJob {  
    status: "running" | "done" | "error";  
}
```

```
result?: BacktestResultPayload;
```

```
errors?: string[];
```

```
config?: BotConfig;
```

```
}
```

```
interface RiskPreset {
```

```
    RISK_PERCENT_PER_TRADE: number;
```

```
    DONCHIAN_PERIOD: number;
```

```
    DONCHIAN_ATR_MULTIPLIER: number;
```

```
    ADX_TREND_THRESHOLD: number;
```

```
    COOLDOWN_BARS: number;
```

```
    MAX_CONCURRENT_TRADES: number;
```

```
    MAX_TOTAL_RISK_PERCENT: number;
```

```
    ENABLE_TRAILING_STOP: boolean;
```

```
    TRAILING_ACTIVATION_ATR_MULTIPLIER: number;
```

```
    TRAILING_DISTANCE_ATR_MULTIPLIER: number;
```

```
}
```

```
interface MetaResponse {
```

```
    default_config: BotConfig;
```

```
    risk_profiles: Record<string, RiskPreset | null>;
```

```
    symbol_groups: Record<string, string[]>;
```

```
    exit_reason_labels: Record<string, string>;
```

```
    timeframes: number[];
```

```
    backtest_periods: string[];
```

```
    strategy_modes: string[];
```

}

۷. قرارداد کامل API

متد	مسیر	بدنه/کونری	توضیح
GET	/api/meta	—	برمی‌گرداند MetaResponse : (تنظیمات پیش‌فرض، پروفایل‌های ریسک، گروه نماد، تایم‌فریم‌ها، حالت‌های استراتژی)
GET	/api/profiles	—	{ profiles: Record<string, BotConfig> }
PUT	/api/profiles/{name}	{ config: BotConfig }	ذخیره/به‌روزرسانی پروفایل
POST	/api/profiles/{name}/duplicate	{ new_name: string }	کپی پروفایل
DELETE	/api/profiles/{name}	—	حذف پروفایل
GET	/api/symbol-groups	—	Record<string, string[]>
PUT	/api/symbol-groups	Record<string, string[]>	ذخیره‌ی گروه‌های نماد
POST	/api/bot/start	{ profile_name: string, config: BotConfig }	شروع اجرای یک پروفایل — پاسخ : { ok: boolean, already_running: boolean }
POST	/api/bot/stop	{ profile_name: string }	توقف پروفایل
GET	/api/bot/status	—	{ bots: Record<string, BotStatus> }
POST	/api/bot/chart/open	{ profile_name: string, symbol: string }	باز کردن چارت یک نماد — پاسخ : { ok: boolean }
GET	/api/report	profile_name, days, start_date?, end_date?	PerformanceReport

متد	مسیر	بدنه/کونری	توضیح
POST	/api/backtest/run	{ config: BotConfig, symbols: string[], period_label: string, start_date?, end_date? }	شروع — Job پاسخ : { job_id: string }
GET	/api/backtest/job/{job_id}	—	BacktestJob
POST	/api/backtest/export/{job_id}	—	فایل خروجی (اکسل) /CSV
GET	/api/log	since: number	{ lines: string[], next: number }
GET	/api/health	—	وضعیت سلامت سرور
GET	/api/dashboard/stats	—	آمار خلاصه‌ی داشبورد
GET	/api/positions/grouped	—	پوزیشن‌های باز، گروه‌بندی‌شده بر اساس پروفایل/نماد
GET	/api/ai-engine/status	symbol, timeframe	وضعیت مدل AI
POST	/api/ai-engine/train	symbol, timeframe, bars	شروع آموزش مدل
POST	/api/v1/analyze	{ symbol, timeframeMinutes, candles: Candle[] }	(داخلی، فقط برای پل — MT5) کندل می‌گیرد، { indicators, decision } برمی‌گرداند

```

interface Candle {
    time: number; // یونیکس ثانیه
    open: number;
    high: number;
    low: number;
    close: number;
    volume?: number;
}

```